

손찬양 경력기술서

Java · Spring Boot 백엔드 개발자

github.com/cyson21 cyson21.github.io/portfolio/

API 개발·운영과 데이터 정합성

2021년부터 백엔드 개발·운영을 담당해 왔으며, Java·Spring Boot를 주력으로 사용합니다.

B2B 리테일 교육 플랫폼과 공공·실시간 데이터 서비스의 API를 개발·운영했습니다. 상태 변경, 다중 서버의 배치 중복 실행, 구매 API 락 경합과 데이터 식별자 오류를 분석하고 서버 로직을 수정했습니다.

회사별 담당 업무

이엠캐스트(주)

2024.03 – 현재 · 백엔드 개발자 · 주임

4인 개발팀에서 20개 이상의 기업 고객 서비스를 제공하는 B2B 리테일 교육 플랫폼의 백엔드 개발·운영을 담당했습니다.

- **교육 플랫폼 API 개발·운영**

Java·Spring Boot로 B2B 리테일 교육 플랫폼의 REST API를 설계·개발·운영했습니다. JPA·QueryDSL·MySQL로 조회·저장을 구현하고 관리자 기능 개편을 지원했습니다.

- **버전 전환·데이터 접근 계층 개선**

플랫폼 2.0 → 3.0 전환의 영향 범위를 점검하고 기존 기능의 회귀를 검증했습니다. 레거시 코드와 API·DB 구조, JPA·QueryDSL 조회·저장 로직을 정리했습니다.

- **운영 오류 분석·수정·회귀 검증**

운영 장애와 데이터 오류를 재현해 원인을 추적하고 API·DB 로직을 수정했습니다. 저장 시점 상태 검증, 분산락 기반 배치 제어와 구매 API 트랜잭션 분리, 복합 식별자 처리와 회귀 테스트를 다뤘습니다.

- **AWS 인프라·CI/CD·배포 운영**

AWS EC2·RDS·Lambda·CloudWatch·WAF 기반 배포·모니터링·운영에 참여했습니다. Docker 배포·전환 이슈에 대응하고 CI/CD 파이프라인 안정화와 코드리뷰 기반 배포 품질 관리에 참여했습니다.

주식회사 화이트스캔

2021.07 – 2024.03 · 백엔드·데이터 개발자 · 연구원

공공·실시간 데이터 기반 서비스의 백엔드와 데이터 처리, 배포·운영을 담당했습니다.

- **도시데이터 수집·가공·조회 API**

서울 실시간 도시데이터 Open API를 수집·가공하는 파이프라인과 조회 API를 구현했습니다.

- **데이터 모델·백엔드 개발**

서비스 요구사항에 맞춰 MySQL·MongoDB 스키마와 REST API를 설계하고, Spring Boot·Django·FastAPI로 데이터 조회·저장 기능을 구현했습니다.

- **예측 결과 연동·Docker 운영**

시계열 예측 결과를 서비스 지표와 기능에 연동하고 관련 서비스를 Docker 컨테이너로 배포·운영했습니다.

대표 실무 문제 해결

이앰캐스트 · B2B 리테일 교육 플랫폼

저장 시점의 상태 변경으로 발생하는 데이터 오류 방지

문제 설정 변경 화면을 연 뒤 관련 작업이 시작되면, 이전 상태를 기준으로 저장하면서 진행 중인 작업에 서로 다른 처리 기준이 적용될 수 있었습니다.

원인 확인 화면 진입 시점의 상태를 저장 시점까지 유효한 값으로 취급하고 있었습니다. 작업 시작 여부를 판단하는 기준도 변경 적용과 후속 데이터 갱신 사이에서 달라질 수 있었습니다.

해결 선택과 이유 저장 요청을 처리하는 서버에서 현재 상태를 다시 확인하는 경계가 필요했습니다. 변경 가능 여부와 후속 갱신이 같은 상태 조회를 사용하도록 맞춰, 화면에 남아 있는 이전 값만으로 변경을 허용하지 않게 했습니다.

변경한 동작 저장 직전 서버에서 현재 상태를 다시 확인하도록 수정했습니다. 화면 진입 후 작업이 시작됐다면 변경을 차단하고, 후속 데이터 갱신 기준도 같은 상태 조회로 통일했습니다.

검증 근거 정상 변경, 저장 전 상태 전이, 기존 이력 존재, 필수 입력 누락을 다루는 테스트를 추가했습니다. 당시 검증 기록에는 단위 테스트 4건의 결과가 남아 있습니다.

[웹 포트폴리오에서 상세 보기](#)

구매 API의 락 경합과 트랜잭션 경계 조정

문제 동시 구매 요청에서 주문 저장 중 DB 데드락이 발생했습니다. 결제 처리를 별도 트랜잭션으로 옮기자 앞 단계의 미커밋 상태를 읽는 문제도 드러났습니다.

원인 확인 주문 저장·컬렉션 갱신의 락 경합에 더해, 결제 처리를 별도 트랜잭션으로 옮긴 뒤에는 앞 단계가 아직 커밋하지 않은 요청 상태를 후속 처리에서 읽는 문제가 있었습니다.

해결 선택과 이유 분산락 적용과 트랜잭션 분리만으로는 상태가 보이는 시점을 맞출 수 없었습니다. 주문 단위로 실행을 제어하면서, 결제 요청 상태가 커밋된 뒤 후속 처리를 호출하도록 경계를 조정했습니다. 재시도는 락 관련 예외에 한정했습니다.

변경한 동작 주문 단위 분산락과 별도 트랜잭션, 락 오류의 제한적 재시도를 적용했습니다. 결제 요청 상태를 먼저 커밋하도록 호출 경계를 조정하고, 재처리 시 결제 완료 상태를 확인했습니다.

검증 근거 동시 요청, 락 예외와 재시도, 결제 완료 주문의 재처리를 다루는 테스트 정의와 후속 수정 기록을 확인했습니다. 초기 QA에서 드러난 상태 가시성 문제도 수정 범위에 포함했습니다.

[웹 포트폴리오에서 상세 보기](#)

대표 실무 문제 해결

이앰캐스트 · B2B 리테일 교육 플랫폼

배치 중복 실행 제어와 실패·실행 누락 모니터링

문제 서버마다 스케줄러가 실행돼 같은 배치가 중복 처리됐습니다. 고객사별로 예정된 배치의 실패와 실행 누락도 구분해야 했습니다.

원인 확인 각 서버에서 독립적으로 실행되는 스케줄러에 공통 실행 제어가 없었습니다. 실행 기록만 확인하면 예정된 작업이 아예 발행되지 않은 경우와 발행 후 결과가 없는 경우도 구분하기 어려웠습니다.

해결 선택과 이유 ShedLock-Redis로 서버 간 실행 주체를 제어했습니다. 이후 별도 개선으로 예정된 고객사·배치 목록을 실제 처리 기록과 대조하는 모니터링을 추가해, 실행 누락도 관측 대상에 포함했습니다.

변경한 동작 ShedLock-Redis 분산락을 적용했습니다. 별도 개선으로 고객사·배치별 상태를 집계하고, 예정 시각 1시간 후 실패·발행 누락·결과 미수신을 구분해 Slack으로 알리도록 구현했습니다.

검증 근거 배치 중복 실행 조치 후 서버 로그와 DB 기록으로 결과를 확인했습니다. 상태·처리 건수·실행 시간을 기록하고 고객사·배치별로 식별할 수 있게 했습니다. 예정 시각 1시간 뒤의 집계와 Slack 알림은 스케줄러·집계 코드 및 관련 테스트 정의로 확인할 수 있습니다.

[웹 포트폴리오에서 상세 보기](#)

복합 식별자 누락으로 발생한 삭제·페이지 집계 오류 수정

문제 같은 사용자의 여러 그룹 데이터를 사용자 ID만으로 구분해 삭제 대상이 누락되고, 조회 제외 조건과 페이지 집계도 실제 행 수와 어긋났습니다.

원인 확인 데이터의 실제 구분 단위는 사용자와 그룹의 조합인데, 조회 제외와 컬렉션 동등성은 사용자 ID만 비교했습니다. 조회 후 애플리케이션 필터링과 별도 함께 계산도 페이지 내용과 총건수를 어긋나게 했습니다.

해결 선택과 이유 실제 행을 구분하는 식별 기준을 조회·삭제·DTO 동등성까지 일관되게 적용했습니다. 필터링을 DB 쿼리로 옮겨 페이지 내용과 전체 건수를 같은 조건으로 계산하도록 맞췄습니다.

변경한 동작 사용자 ID와 그룹 유형의 복합 식별자를 조회·삭제·컬렉션 동등성에 적용했습니다. 제외 조건을 DB 쿼리로 옮기고 페이지 집계를 맞췄습니다.

검증 근거 같은 사용자의 서로 다른 그룹을 Set에서 보존하는 경우, 선택한 조합만 삭제하는 경우, 일부 그룹 제외와 페이지 경계를 다루는 테스트 및 당시 검증 기록이 있습니다.

[웹 포트폴리오에서 상세 보기](#)

개인 프로젝트의 코드·테스트 근거는 cyson21.github.io/projects/에서 확인할 수 있습니다.